

El Problema

Consumo Excesivo de RAM:

Instanciar miles o millones de objetos similares replica datos idénticos en cada instancia, agotando la memoria.

Ejemplo: 100,000 árboles cargando cada uno la misma textura, malla y color de forma redundante.

La Solución Flyweight

Divide el estado de los objetos en dos categorías para compartir la información inmutable a través de un pool centralizado:

Estado Intrínseco

Inmutable y Compartible.

Se almacena dentro del

Flyweight.

(Ej: Textura, Color, Malla)

Estado Extrínseco

Único por Instancia.

Se pasa como **parámetro**.

(Ej: Posición X, Y, Escala)

Estructura y Participantes

Flyweight (Interfaz)

Declara la interfaz para recibir el estado extrínseco.

ConcreteFlyweight

Almacena el estado intrínseco compartido de forma inmutable.

FlyweightFactory

Gestiona el pool de objetos. Reutiliza los existentes o crea nuevos.

Client (Cliente)

Mantiene el estado extrínseco y lo pasa al Flyweight al operar.

Pros y Contras

Ventajas:

- Reducción masiva de RAM.
- Mayor escalabilidad.

Desventajas:

- Mayor complejidad.
- El cliente gestiona el estado externo.

Caso Práctico: Bosque de 100,000 Árboles

En lugar de instanciar 100,000 objetos completos, se genera **un solo Flyweight** por tipo de árbol. La posición de cada uno se maneja externamente.

Hasta 99% Menos Consumo de RAM

Implementación en Python:

```
class ArbolFlyweight: # Estado Intrínseco
    def __init__(self, tipo, color, textura):
        self.tipo = tipo
        self.color = color
        self.textura = textura

    def dibujar(self, x, y): # Estado Extrínseco
        como parametro
        print(f'{self.tipo} en ({x},{y})')

class ArbolFactory:
    _pool = {}
    @classmethod
    def get(cls, tipo, color, textura):
        key = (tipo, color, textura)
        if key not in cls._pool:
            cls._pool[key] = ArbolFlyweight(tipo,
            color, textura)
        return cls._pool[key]

# Cliente: 1 solo flyweight real en RAM para
100,000 instancias
bosque = [
    (ArbolFactory.get('Pino', 'verde', 'tex.png'),
    x, x % 500)
    for x in range(100_000)
]

for arbol, x, y in bosque:
    arbol.dibujar(x, y)
```

Bibliografía (Normas APA)

- Gamma, E., Helm, R., Johnson, R., y Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.
- Refactoring.Guru. (2024). *Flyweight*. Recuperado de <https://refactoring.guru/es/design-patterns/flyweight>
- SourceMaking. (2024). *Flyweight design pattern*. Recuperado de https://sourcemaking.com/design_patterns/flyweight